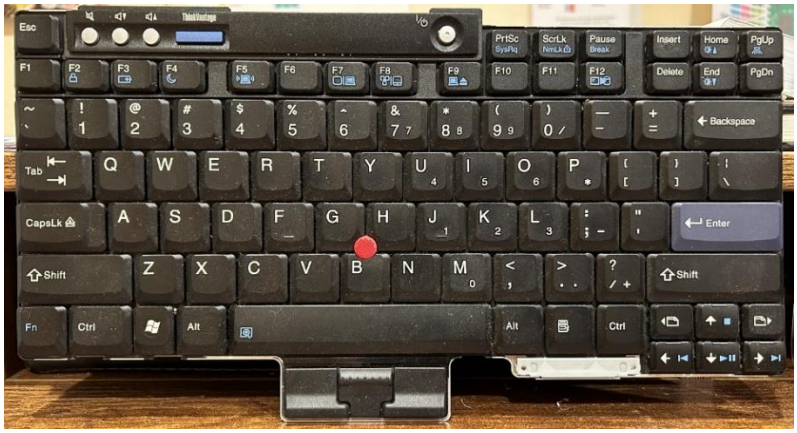
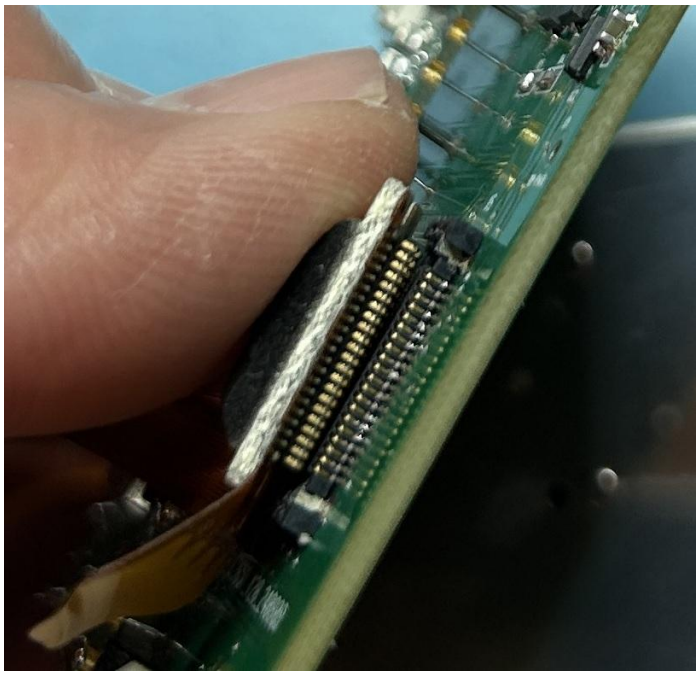
Raspberry Pi Pico W Keyboard Controller for the Lenovo T61 and T420 - 8/4/2026

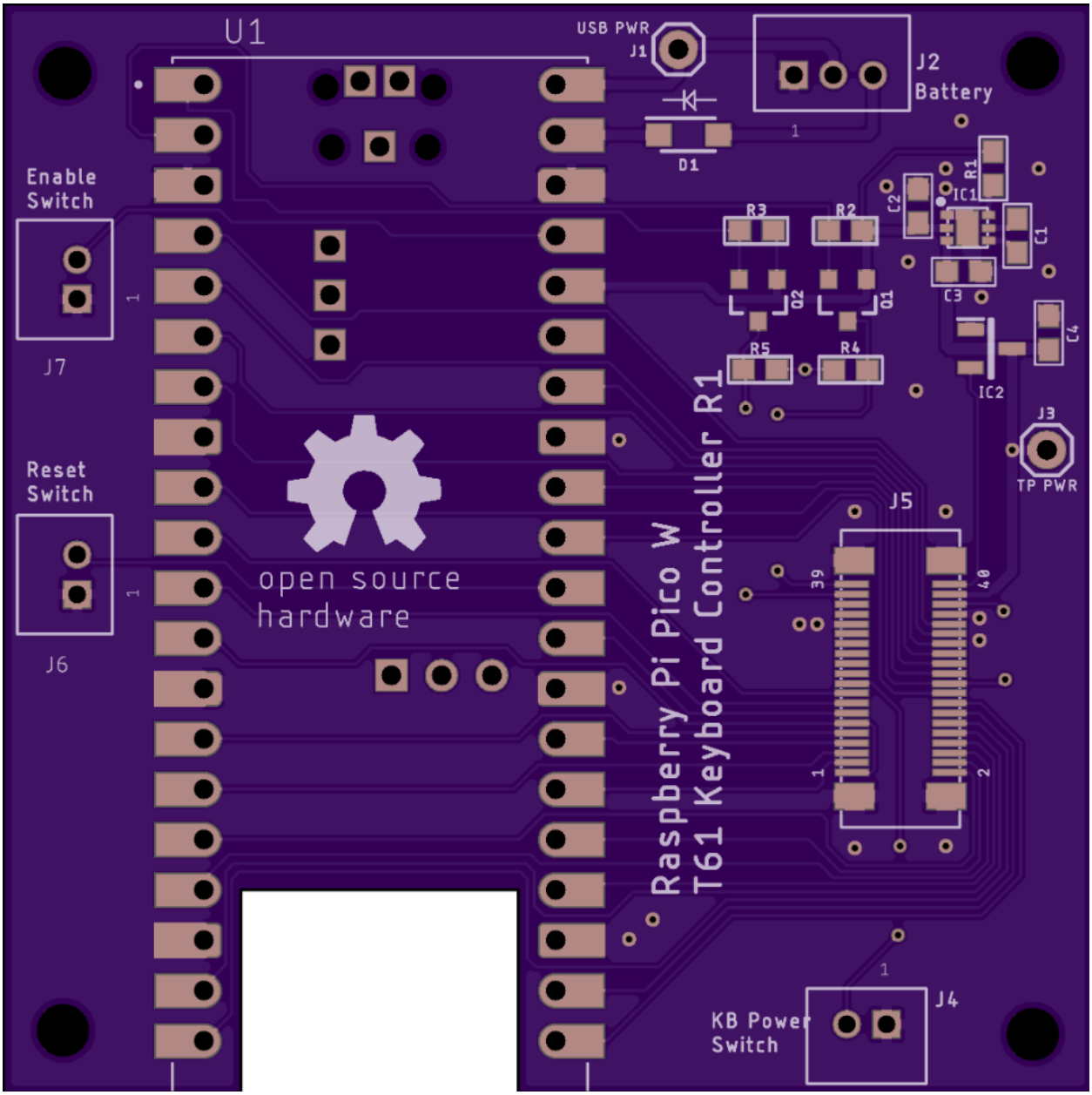
This document will describe a Raspberry Pi Pico W USB and Bluetooth Low Energy (BLE) controller for the Thinkpad T60 series keyboards like the T61 shown below. The T400 series keyboards are also supported using a slightly different circuit board. KMK and QMK USB code will be described along with Arduino BLE code. All code is available at my Github repository along with the Eagle files and zipped Gerber files that can be sent directly to a board manufacturer.



The 44 pin connector soldered to the end of the FPC cable has 4 corner pins that are not used when installed in the 40 pin board connector. The board connector part number is AA01B-S040 and is available from Ali-Express. The picture below shows how to manually align the contacts so there is an extra contact on each end.

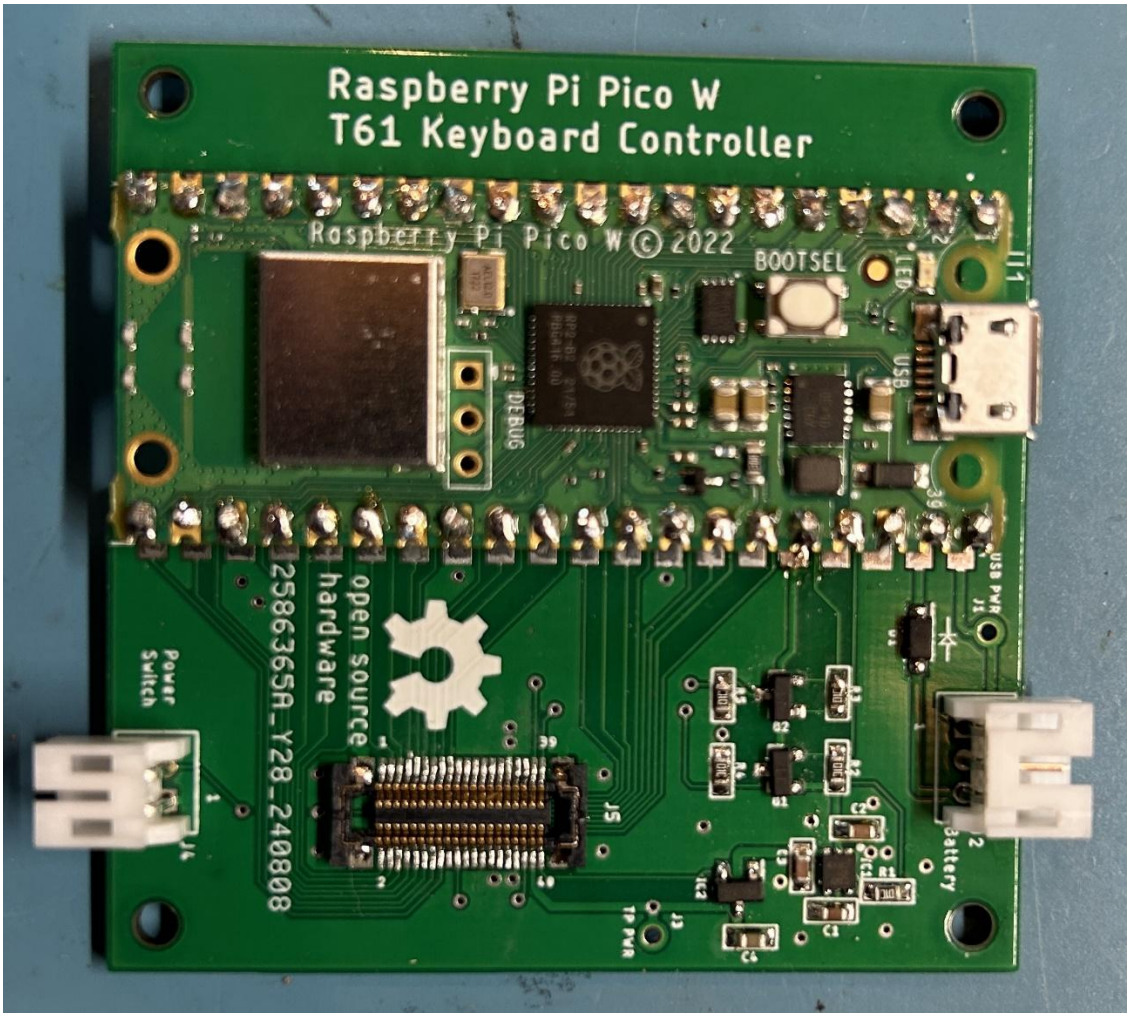


The GPIO pins of the Raspberry Pi Pico are connected to the 40 pin AA01B-S040 connector on the circuit board shown below. The other surface mount components provide level translation, reset generation, and 5 volt power for the TrackPoint.

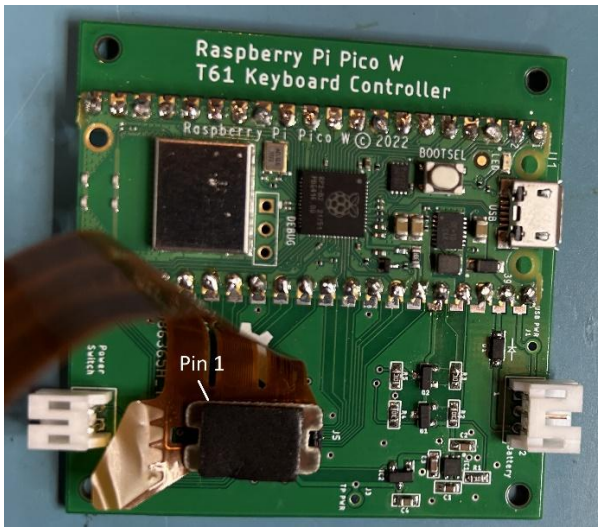



The cutout in the board gives better reception for the Bluetooth antenna on the Pico. The Pico can be mounted with header pins or soldered directly to the board for a lower profile. The Eagle file "Pico_T61_Keyboard_R1.brd" or "Pico_T420_Keyboard_R1.brd" can be sent to OSH Park or Eurocircuits. The Gerber file "Pico_T61_Keyboard_R1.zip" or "Pico_T420_Keyboard_R1.zip" can be sent to any fab house such as JLCPCB or PCBWay. The layout and schematic files are in the Eagle directory at my repo. Hand soldering the keyboard connector with an iron is challenging. I prefer applying solder paste to the surface mount pads using a stencil. You can use a hot plate and heat gun to melt the solder but a reflow oven, such as the converted toaster oven described here, makes the job much easier.

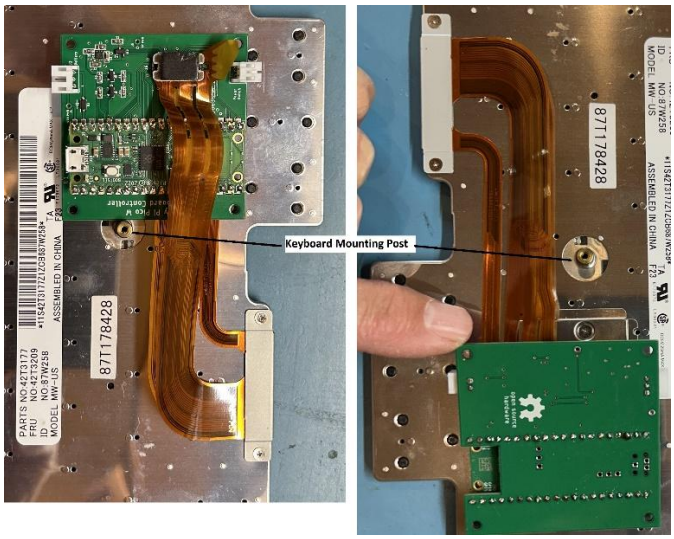
The picture below shows the assembled (original) T61 board that was manufactured by JLCPCB.




There is no keying so it's possible to rotate the keyboard connector 180 degrees. See the picture below for the correct orientation of pin 1 on the FPC cable connection.



The circuit board can either be mounted facing up or down as shown below. The keyboard mounting post clears the board in both cases.



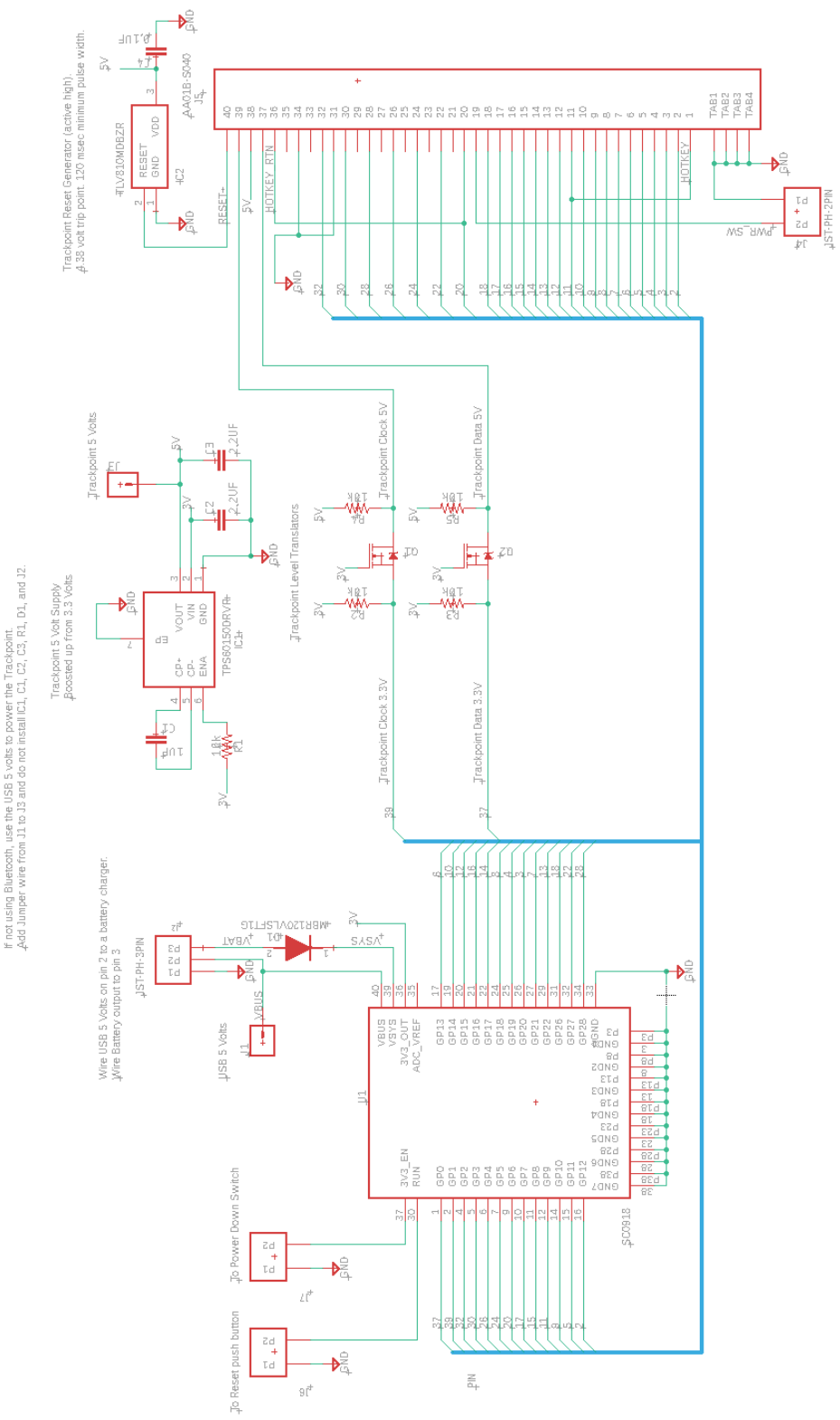

The Eagle schematic “Pico_T61_Keyboard_R1.sch” is in the Eagle Layout folder at my repo and shown on the next page. In the same folder is the Eagle schematic “Pico_T420_Keyboard_R1.sch” which changes the HOTKEY_RTN from pin 36 to pin 34. Everything else is the same as the T61. The trackpoint clock and data GPIO pins had to be swapped for the QMK library. That’s why everything has R1 in the name.

The R1 board added two JST connectors for wiring to a push button reset switch and an Enable switch.

The TrackPoint circuitry in the keyboard is powered with 5 volts and draws about 6ma. The Pico GPIO’s are not 5 volt tolerant so there are BSS138 N channel FETs labeled Q1 and Q2 that act as level translators for the TrackPoint Clock and Data signals. The TrackPoint reset signal can be created with a resistor and capacitor but to be more reliable, I’ve included a TLV810MDBZR reset generator labeled IC2. It creates a 120msec active high reset pulse at 4.38 volts. A 2 pin JST connector at location J4 can be installed if you have an off-board power latch circuit that will be controlled by the keyboard’s “Power” button.

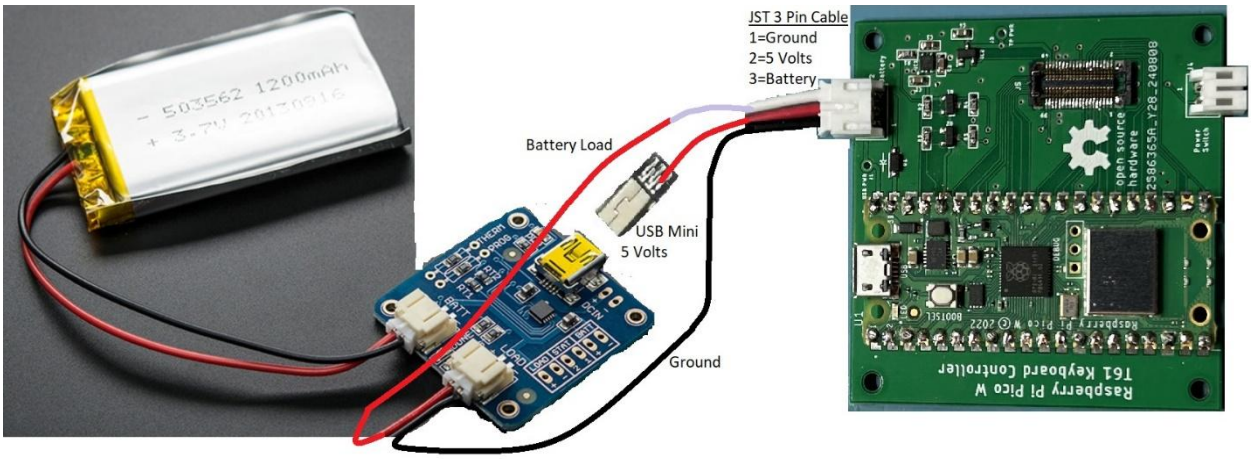
Bluetooth operation implies the board will be running from a lithium battery which ranges from 3 to 4.2 volts. When the battery needs charging, a USB cable is connected to the PICO from a power source. Bluetooth operation can continue while the battery is charging. USB 5 volts is “or tied” with the battery voltage via Schottky diodes. These diodes feed the Pico’s internal 3.3 volt buck/boost regulator. One diode is internal to the Pico and the other is labeled D1 on the board. I’ve routed the Pico’s 3.3 volts to a TPS60150 switched capacitor charge pump labeled IC1 that creates the 5 volts to power the TrackPoint and level translators.

If the keyboard will only be used for USB, solder a jumper wire between test points J1 and J3. This routes USB 5 volts directly to the TrackPoint. Do not install charge pump IC1 or its associated components, R1, C1-C3. Do not install Schottky diode D1 or the 3 pin JST connector at J2. You will still need the PS/2 level translators at Q1, Q2, R2-R5, for the TrackPoint clock and data signals.

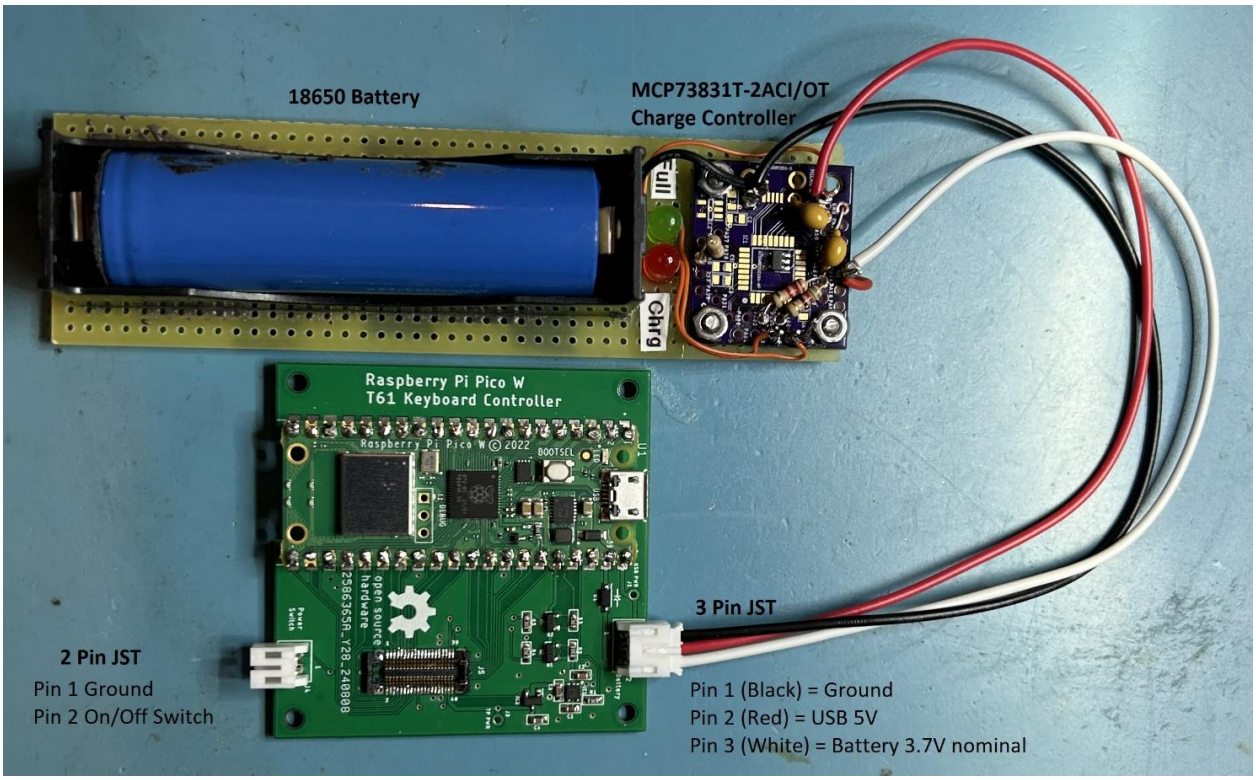


Schematic for Thinkpad T61 keyboard controller (R1)

The 3 pin JST connector labeled J2 on the board is cabled to the battery and charging circuit. Pin 1 on the JST connector is ground, pin 2 is USB 5 volts, and pin 3 is the battery output. This picture shows the cable connections using Adafruit components. Their charger board needs a USB mini connector to feed in 5 volts.




Instead of buying a lithium battery and charging circuit, I put my own together using an MCP73831T-2ACI/OT controller chip and battery salvaged from a laptop (see below).



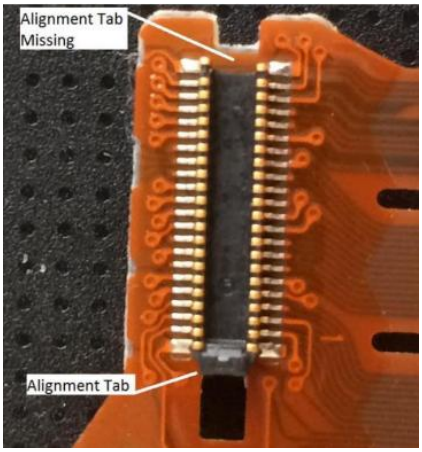
Parts list with supplier links for the Assembled Circuit Board is given below.

| Qty | Part Number | Ref | Description |
|---|---|---|---|
| 1 | KGM15BR71H104JM | C4 | 0.1uF 0603 X7R Ceramic Chip Capacitor 50V 10% |
| 5 | RC0603JR-0710KL | R1-R5 | 10K 0603 5% tolerance 1/10W Chip Resistor |
| 1 | KGM15AR71C105KT | C1 | 1 uF 0603 X7R Ceramic Chip Capacitor 16V 10% |
| 2 | KGM15BR51A225KM | C2, C3 | 2.2 uF 0603 X5R Ceramic Chip Capacitor 10V 10% |
| 2 | BSS138 | Q1, Q2 | BSS138 N-Channel MOSFET SOT23-3 50V 230ma |
| 1 | AA01B-S040 | J5 | 40 Pin Keyboard Connector |
| 3 | S2B-PH-K-S | J4,J6,J7 | 2 Pin right angle JST Board Connector 2mm |
| 1 | S3B-PH-K-S | J2 | 3 Pin right angle JST Board Connector 2mm |
| 1 | MBR120VLSFT1G | D1 | Schottky Diode 20V 1A |
| 1 | Raspberry Pi Pico W | U1 | Raspberry Pi Pico W with or without Header pins |
| 1 | TLV810MDBZR | IC2 | Reset Generator Active High 4.38V 120ms Push-Pull SOT23-3 |
| 1 | TPS60150DRVR | IC1 | 5V 140ma switched capacitor voltage converter WSON-6 package |
| 1 | Pico_T61_Keyboard_R1 | PWB | Printed Wiring Board manufactured using Eagle file “Pico_T61_Keyboard_R1.brd” or Gerber file “Pico_T61_Keyboard_R1.zip” |

The total parts cost is about $90 if using OSHPark or about $75 if using JLCPCB (with slow shipping). A lithium battery, charging circuit, USB Mini connector, and 3 pin JST cable will cost about $50.

OSH Park sends you 3 boards and JLCPCB sends you 5 boards so build this project with friends to spread out the cost. You will save a few bucks if you order the Pico without header pins. You can install your own headers or solder the Pico directly to the board. I installed header pins on the grounds for a robust connection and 30 gauge wires on GPIO’s so I could reroute signals for debug.

The board has surface mount pads for the keyboard connector that are extra-long to accept an alternate connector, Digikey part number WM6787CT-ND. This was done just in case the AA01B-S040 is not available. I’ve used the WM6787CT-ND connector on previous T61 projects and it works fine. It lacks the plastic guides on each end so you must visually align the pins when mating with the keyboard connector. Even the AA01B-S040 needs help with alignment if the plastic guides on the keyboard connector have broken off (see below). The guides break off if you’re way off when pushing it into the board connector (my bad).



The T61 keyboard connections to the Pico GPIO's and other devices are shown below.

| Lenovo ThinkPad T61 FPC Connector | Pico GPIO number | Notes | T61 Schematic Signal Name |
|---|---|---|---|
| 1 | 9 | Added to key matrix | HOTKEY |
| 2 | 12 | | DRV<4> |
| 3 | 20 | | SENSE<5> |
| 4 | 19 | | DRV<5> |
| 5 | 11 | | SENSE<0> |
| 6 | 13 | | DRV<8> |
| 7 | 21 | | SENSE<3> |
| 8 | 18 | | DRV<6> |
| 9 | 10 | | SENSE<2> |
| 10 | 14 | | DRV<3> |
| 11 | 9 | | SENSE<4> |
| 12 | 15 | | DRV<7> |
| 13 | 22 | | SENSE<1> |
| 14 | 17 | | DRV<2> |
| 15 | 8 | | SENSE<6> |
| 16 | 16 | | DRV<10> |
| 17 | 7 | | SENSE<7> |
| 18 | 26 | | DRV<1> |
| 19 | | PWR SW – J4 Pin 2 | PWR SW |
| 20 | 6 | | DRV<9> |
| 21 | No connect | | NC |
| 22 | 27 | | DRV<0> |
| 23 | No connect | | NC |
| 24 | 5 | | DRV<11> |
| 25 | No connect | | KBDID0 |
| 26 | 4 | | DRV<14> |
| 27 | No connect | | KBDID1 |
| 28 | 28 | | DRV<12> |
| 29 | No connect | | KBDID2 |
| 30 | 3 | | DRV<15> |
| 31 | Pico GND | | KBDID RTN |
| 32 | 2 | | DRV<13> |
| 33 | No connect | | NC |
| 34 | Pico GND | | KBDID RTN |
| 35 | No connect | | NC |
| 36 | 6 | Added to key matrix | HOTKEY RTN |
| 37 | 0 | Level Translated | TP_DATA |
| 38 | | 5 volts from TPS60150 | TP_5V |
| 39 | 1 | Level Translated | TP_CLK |
| 40 | | TLV810 Reset Generator | TP_RESET |

Note1: The T420 keyboard uses pin 34 as the HOTKEY_RTN instead of 36.

Note2: The original PCB connected 37 to GPIO 1 & 39 to GPIO 0. The trackpoint wouldn't work for QMK.

The Pico inputs are the Sense<0> through <7> columns across the top programmed with pullups. The Pico outputs are the Drive<0> through <15> rows on the side. These are programmed to act like an open drain output. They drive a logic low and when supposed to float, they are programmed to be an input.

Matrix for the Lenovo ThinkPad T61 Keyboard

| **Pico GPIO Number** | **Sense<0> GPIO 11** | **Sense<1> GPIO 22** | **Sense<2> GPIO 10** | **Sense<3> GPIO 21** | **Sense<4> GPIO 9** | **Sense<5> GPIO 20** | **Sense<6> GPIO 8** | **Sense<7> GPIO 7** |
|---|---|---|---|---|---|---|---|---|
| **Drive<0> GPIO 27** | Back-Tick | 1 | Q | Tab | A | Esc | Z | |
| **Drive<1> GPIO 26** | F1 | 2 | W | Caps-Lock | S | | X | |
| **Drive<2> GPIO 17** | F2 | 3 | E | F3 | D | F4 | C | |
| **Drive<3> GPIO 14** | 5 | 4 | R | T | F | G | V | B |
| **Drive<4> GPIO 12** | 6 | 7 | U | Y | J | H | M | N |
| **Drive<5> GPIO 19** | Equal | 8 | I | Right-Brace | K | F6 | Comma | |
| **Drive<6> GPIO 18** | F8 | 9 | O | F7 | L | | Period | |
| **Drive<7> GPIO 15** | Minus | 0 | P | Left-Brace | Semi-colon | Quote | | Forward-Slash |
| **Drive<8> GPIO 13** | F9 | F10 | | Back-Space | Back-Slash | F5 | Enter | Space |
| **Drive<9> GPIO 6** | Insert | F12 | | | **Fn added here** | | | Arrow-Right |
| **Drive<10> GPIO 16** | Delete | F11 | Volume-Up | Volume-Down | Mute | Think-Vantage | | Arrow-Down |
| **Drive<11> GPIO 5** | Page-Up | Page-Down | **GUI** | | Menu | | Page-Left | Page-Right |
| **Drive<12> GPIO 28** | Home | End | | | | Arrow-Up | Pause | Arrow-Left |
| **Drive<13> GPIO 2** | | Print-Screen | Scroll-Lock | | | **Alt-L** | | **Alt-R** |
| **Drive<14> GPIO 4** | | | | **Shift-L** | | | **Shift-R** | |
| **Drive<15> GPIO 3** | **Cntrl-L** | | | | | | **Cntrl_R** | |

The keyboard uses 24 Pico GPIO's and the TrackPoint uses the remaining 2 GPIO's. Normally the Fn signal (known as HOTKEY) is received on a dedicated GPIO and the HOTKEY_RETURN is tied to ground. Since there are no spare GPIO's, the HOTKEY was wired into the matrix at a blank spot as shown above. This requires the HOTKEY_RETURN to not be tied to ground. The T60 series keyboards have HOTKEY_RETURN on connector pin 36 but on the T400 series keyboards it is on pin 34. This is why there are two separate circuit boards depending on which keyboard you are using.

Software and Testing:

**KMK** was used to verify the keyboard connections but it doesn't support the PS/2 interface to the TrackPoint. The KMK Getting Started page describes how to install CircuitPython, the KMK folder, and code.py on the Pico. Several KMK websites describe how to modify the code for a keypad. Note that some websites use main.py instead of code.py as the file name for your code. Both names will work but code.py is the preferred name for CircuitPython. I expanded the keypad example code into a matrix for a T61 keyboard. The code.py for the T61 at my repo is fully functional as a USB keyboard. The code switches to a second layer from the main layer when the Fn key is held down. CircuitPython (used by KMK) lacks a complete Bluetooth stack implementation for the Pico W chipset so KMK can't be used as a BLE controller.

**QMK** code was compiled for the keyboard and trackpoint. The keyboard.json and keymap.c files were created using the procedure described in Step 17 of my Instructable. The code switches to a second layer from the main layer when the Fn key is held down. The code has a third layer for the embedded number pad that is activated when the Num Lock key is selected (regardless of Fn). The trackpoint code in QMK uses the PIO in the Pico and requires the Data GPIO pin to be before the Clock GPIO pin. I had to swap the Clock and Data header connections to make my original board work. The R1 board at my repo fixes this problem. The QMK keyboard.json and keymap.c files are at my repo as well as my compiled Lenovo_t61_default.uf2 file. You could build this board and flash it with my .uf2 file, eliminating the need to download the QMK compiler. Any modifications would require compiling a new .uf2 file as described in my Instructable. Note that QMK does not natively support the Pico W Bluetooth chipset so it can't be used as a BLE controller.

**Arduino USB C++** code was created by Claude that uses the Earle Philhower Pi Pico core and the Adafruit TinyUSB library. The keyboard, trackpoint, and combined keyboard and trackpoint files listed below are at my repo:

"Pico_T61_KB_USB.ino" only controls the keyboard

"Pico_T61_TP_USB.ino" only controls the trackpoint

"Pico_T61_KBTP_USB.ino" controls both the keyboard and backlight

**Bluetooth Low Energy (BLE) C++** code for the T61 uses the Earle Philhower Pico libraries. The keyboard, trackpoint, and combined keyboard and trackpoint BLE files are at my repo. In the Arduino IDE, set the board to Raspberry Pi Pico W and the IP/Bluetooth Stack to IPV4 + Bluetooth. Each time you re-flash the Pico, you need to "Remove Device" in your computer or phone's Bluetooth settings and then pair it again. Gemini AI helped me get the keyboard and trackpoint code working. Claude improved the code further to make it more robust. There are 3 different T61 Arduino files at my repo:

"Pico_W_T61_KBTP_BLE.ino" controls the keyboard and trackpoint. Use this file unless debugging a problem.

"Pico_W_T61_KB_BLE.ino" controls the keyboard only. Use this if trying to isolate a problem with the combined code above.

"Pico_W_T61_TP_BLE.ino" controls the trackpoint only. Use this if trying to isolate a problem with the combined code above.

The combined keyboard and trackpoint code uses the dual-profile definition "KeyboardAndMouseBLE" so your computer detects the Pico W as a single composite desktop device (a unified Keyboard + Mouse) over a single Bluetooth connection. Gemini and Claude say this code will run on a Pi Pico 2 W (I haven't tried it) as long as you select the Raspberry Pi Pico 2W board under the tools menu.

Battery Operation:

I've been able to type and use the trackpoint over BLE while running on battery power. The battery drains in about 3 days when connected continuously to a PC via BLE. I added a connector so Enable pin 37 can be grounded which puts the Pico to sleep. Hooking up the USB cable provides 5 volts and the keyboard can be used while charging. The Pico needs to be reset when the battery is drained in order to get the code running again (that's why I added a connector for a reset). Tie Run/Reset pin 30 to a normally open push button switch so you can ground this pin. An alternative is to tie the Pico's Reset pin (pin 30) to the power on/off button on the keyboard at J4 Pin 2.